

Benchmark: Lazy Loading Container Images for AI/ML on Kubernetes

Hoa Ngo · GreenNode · KubeCommunity 2026

Outline

1. The cold-start problem
2. Three lazy-loading technologies
3. How a snapshotter reads / writes
4. Benchmark — Jupyter & vLLM, internal vs external registry
5. Deploying lazy loading in production

Why this matters

- AI/ML container images are large: vLLM **8.3 GB**, Jupyter+PyTorch **3.6 GB**
- Traditional OCI: pull the **entire** image before the container starts
- Cold starts of 100s+ stall autoscaling and leave GPUs idle
- Lazy loading starts the container from a **small metadata index** and fetches data chunks **on-demand**

Three lazy-loading technologies

- **Nydus** — RAFS + FUSE, on-demand 4 MB chunks (Dragonfly project)
- **SOCI** — AWS Seekable OCI, zTOC index over existing layers, parallel fetch
- **Stargz** — Google eStargz, TOC-based, per-file granularity

All three need a snapshotter on the node and (for Nydus/Stargz) a one-time image conversion.

How a file read flows through the stack

```
container reads /opt/conda/.../torch/__init__.py
|
overlayfs      upperdir miss → fall through to lowerdir (FUSE)
|
RAFS FUSE      kernel forwards to nydusd
|
nydusd         image.boot lookup → chunk ID → cache miss
|
HTTPS          GET harbor/.../<sha>.blob    (4 MB)
|
cache          write to /var/lib/containerd-nydus/cache
|
return         bytes flow back up – container never knows
```

First read: network. Subsequent reads: local cache. Cache survives pod restarts and shares across pods on the same node.

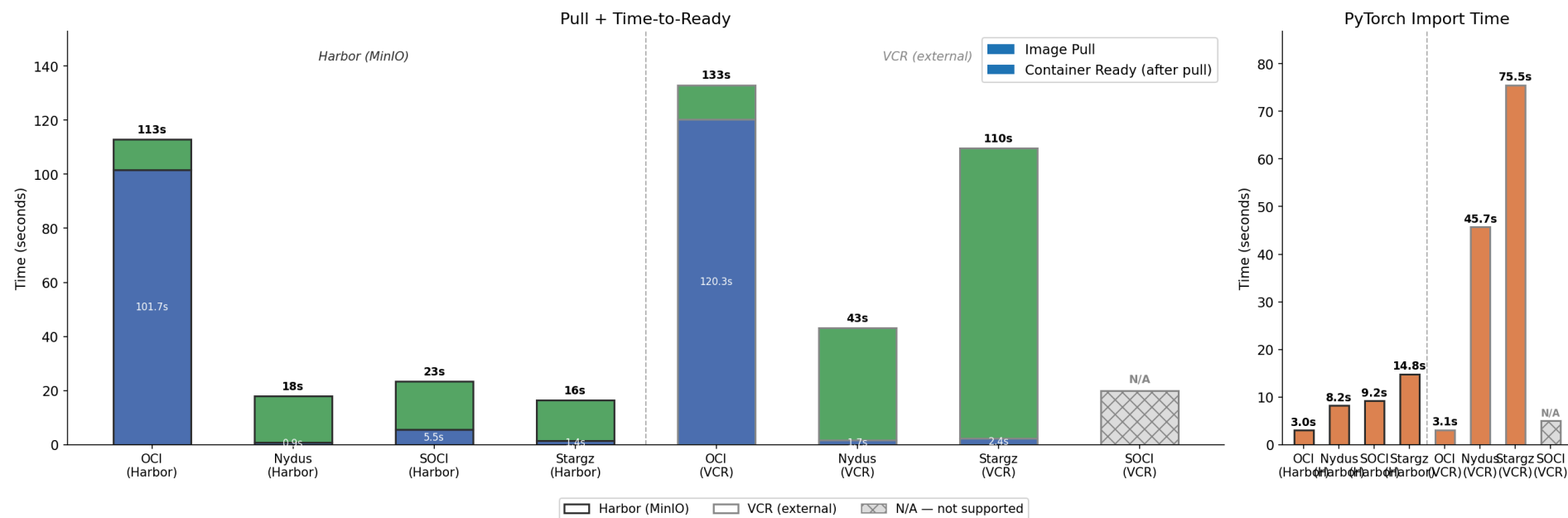
Test environment

Node spec	8 vCPU / 16 GB / SSD / 10 Gbps
OS	Ubuntu 24.04, kernel 6.8, containerd
Images	<code>jupyter-pytorch-cuda-full:v1.9.2</code> (3.6 GB), <code>vllm-openai:v0.12.0</code> (8.3 GB)
Registries	Harbor + MinIO (internal) · Cloud registry

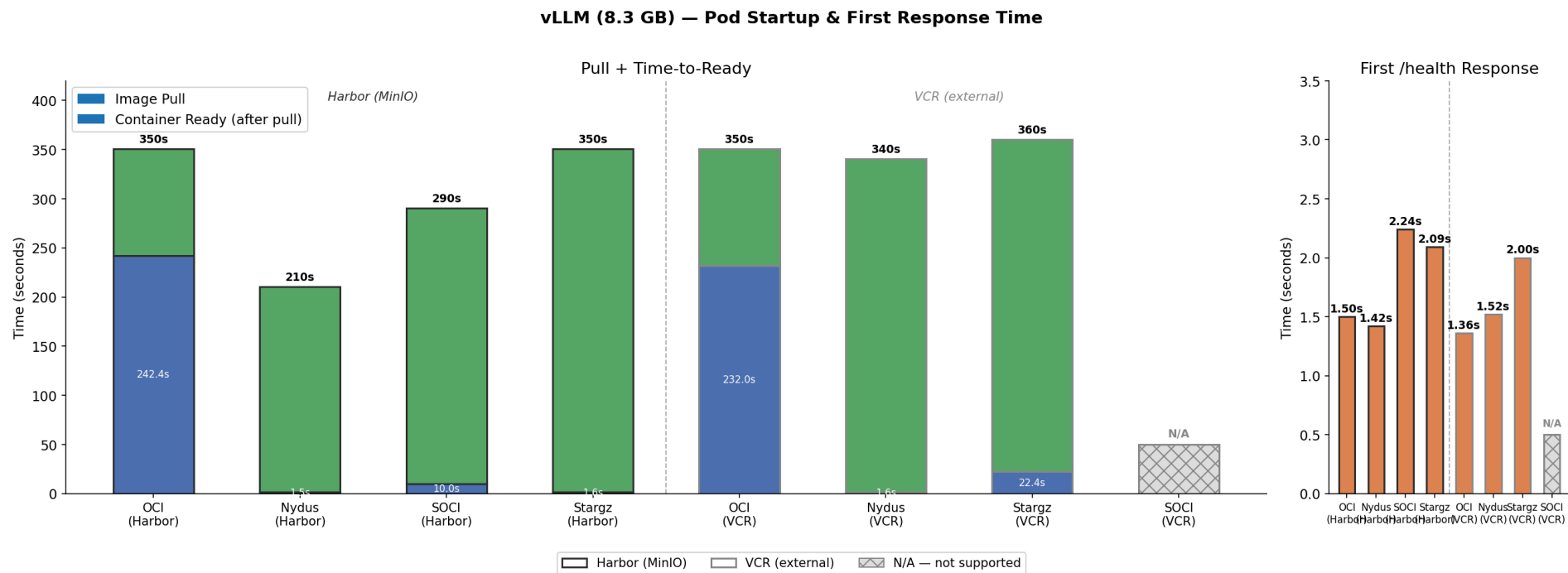
Measured per run: image pull · time-to-ready · first app metric (`import torch` / first `/health`).

Jupyter PyTorch (3.6 GB) — results

Jupyter PyTorch (3.6 GB) — Pod Startup & PyTorch Import Time



vLLM (8.3 GB) — results



Why vLLM gets a smaller win

- vLLM loads almost all Python libraries at startup → most chunks get fetched anyway
- Lazy loading degrades to **parallel fetch with smaller working set** for "import everything" workloads
- Nydus still saves **140s/node** (350s → 210s) — real, just not 6×
- SOCI 290s vs Stargz 350s on Harbor: larger chunk granularity → fewer round trips

Registry can be a the bottleneck

- Nydus + Harbor (PyTorch import): **8s**
- Nydus + VCR (PyTorch import): **45s**
- Same image, same snapshotter — only the registry moved
- On-demand chunks = many small HTTPS GETs in the hot path; $RTT \times \text{chunks} = \text{real seconds}$
- Observation: Some hypervisor registry (gcp/aws) that thing might not happend. but you pay extra fee for throughput. Still needed.

Do we need help of the cloud provider?

Lazy loading needs **two node-level injections**:

```
containerd → snapshotter = "nydus" + register proxy_plugins.nydus  
kubelet    → --image-service-endpoint=.../containerd-nydus-grpc.sock
```

- **Self-managed nodes** (custom image / cloud-init): you own both — straightforward
- **Managed K8s**: depends on what the provider exposes
 - Launch templates, node user-data, kubelet extra-args → workable (EKS/GKE-style)
 - Fully managed kubelet, no node bootstrap hook → blocked
 - Snapshotter must start **before kubelet**

Audit your cloud's support before committing to a rollout.

Image conversion: who runs it?

Lazy-loading images need a one-time format conversion. Two models:

- **Registry-side (server does it)**
 - Harbor with the nydusify plugin → push `:v1.0` , get `:v1.0-nydus` auto
 - Alibaba ACR / VolcEngine (ByteDance) → native nydus on-demand
 - AWS ECR → native SOCI (v2 default since 2026)
 - GCP Artifact Registry → GKE Image Streaming (their own, not SOCI)
- **Client-side (you do it in CI)**
 - A `nerdctl image convert --nydus` step in the build pipeline
 - Or a CronJob/webhook watching for new upstream tags
 - ~2–5 min per image, reused fleet-wide

Either way: workloads pin `<tag>-nydus` . Rollback = swap tag.

Check if your registry supports server-side conversion before building your own.

Recommendation & takeaways

Pick Nydus for AI/ML on Kubernetes today.

Pod ready, Jupyter	113s → 17.9s (6.3×)
Pod ready, vLLM	350s → 210s (1.7×)

- **SOCI** if your image lacks a giant layer (torch + CUDA) — OCI-compliant, no rebuild.
- **Stargz** is the awkward middle child.
- **Registry locality is non-negotiable** — co-locate blobs with workers.
- **Run the snapshotter as systemd**, not as a Pod.
- **Audit your cloud** for kubelet/containerd injection + registry-side conversion.



Full report & repo: github.com/cloud-guru/CNCF-HCM-2026